

과목명: 컴파일러 구성 (CSE4120)

학번: 20211605

이름: 하지훈

사용 LLM 모델: Google Gemini

1. 개요

이번 과제는 Mini-C의 AST(Abstract Syntax Tree)를 입력받아, 이를 실행 가능한 중간 표현인 IR(Intermediate Representation)로 변환하는 Code Generator를 구현하는 것이었다.

Project #1과 달리 이번 단계에서는 배열(Array)과 포인터(Pointer) 연산이 추가되었고, 단락 평가(Short-circuit evaluation)를 구현해야 했다. 특히 C언어 특유의 '배열의 포인터 변환(Array Decay)' 규칙과 '포인터 산술 연산(Pointer Arithmetic)' 시 주소 스케일링(Scaling)을 IR 명령어로 풀어내는 과정에서 논리적인 어려움이 있었다.

또한, And와 Or 연산자가 단순 산술 연산이 아니라 제어 흐름(Control Flow)을 변경해야 한다는 점을 구현하는 데 막힘이 있어 LLM(Gemini)에 질의하여 힌트를 얻고 코드를 완성했다. 아래는 프로젝트를 수행하며 입력한 Prompt와 LLM의 답변, 그리고 이를 통해 내가 코드를 어떻게 이해하고 수정했는지를 정리한 내용이다.

2-1. 내가 입력한 Prompt (포인터 연산과 타입 추론 문제)

transExp 함수에서 Add 연산을 구현 중이야. 일반적인 정수 덧셈(int + int)은 그냥 AddOp를 쓰면 되는데, 포인터 연산(int* + int)의 경우 주소값을 단순히 1 더하는 게 아니라 sizeof(int)만큼 곱해서 더해야 하잖아.

그런데 AST.Add 노드만 보서는 피연산자가 정수인지 포인터인지 알 수가 없어. transExp 내부에서 피연산자의 타입을 미리 알 수 있는 방법이나 헬퍼 함수 구조를 추천해줘. 코드를 다 짜주지는 말고 구조만 설명해 줘.

2-2. LLM이 출력한 답변

아주 중요한 포인트에 도달하셨군요. AST.Add 노드 자체에는 타입 정보가 없어서, 이게 단순 정수 덧셈인지 포인터 연산인지 구별하기가 어렵다는 점이 바로 이번 과제의 핵심 난관 중 하나입니다.

이 문제를 해결하기 위해서는 *****현재 심볼 테이블을 참고하여 식(Expression)의 타입을 역추적하는 헬퍼 함수*****를 하나 만드시는 것이 가장 좋습니다.

구조를 잡아드릴게요!

1. 타입 추론 헬퍼 함수 (getTyp) 만들기

transExp 내부나 외부에, 현재의 symtab과 Exp를 받아서 그 식이 어떤 CType인지를 반환해주는 재귀 함수를 정의하세요.

함수 시그니처 (예시):

F#

// 식 e가 어떤 타입(int, int*, bool 등)인지 반환하는 함수

let rec getTyp (symtab: SymbolTable) (e: Exp) : CType = ...

구현 논리:

- Num _: 당연히 CInt입니다.
- Boolean _: 당연히 CBool입니다.
- Var x: 여기가 핵심입니다. symtab에서 x를 찾아서 그 타입을 반환합니다.
 - (팁: 만약 int arr[3] 같은 배열이라면, 식에서 사용될 때 int*로 취급되어야 합니다. 이 부분 처리를 여기서 해주면 편합니다.)
- Add (e1, e2) / Sub (e1, e2):
 - e1이나 e2의 타입을 재귀적으로 확인(getTyp)합니다.
 - 둘 중 하나라도 포인터 타입(CIntPtr, CBoolPtr)이면, 결과도 포인터 타입입니다.
 - 둘 다 정수면 결과도 CInt입니다.
- Deref e (*e):
 - e의 타입을 확인해서, CIntPtr이면 CInt를, CBoolPtr이면 CBool을 반환합니다.

2. transExp의 Add 케이스에서 활용하기

이제 transExp 함수 안에서 Add (e1, e2)를 만났을 때, 방금 만든 getTyp을 활용하면 됩니다.

구조 (의사 코드):

F#

| Add (e1, e2) ->

// 1. 먼저 e1, e2를 각각 IR로 변환해둡니다 (레지스터 r1, r2 확보)

let (r1, i1) = transExp symtab e1

let (r2, i2) = transExp symtab e2

// 2. 헬퍼 함수로 타입을 확인합니다

let t1 = getTyp symtab e1

let t2 = getTyp symtab e2

// 3. 타입 조합에 따라 다른 IR을 생성합니다

match (t1, t2) with

| (포인터, 정수) ->

// 포인터 연산입니다!

// 정수 쪽(r2)에 데이터 크기(예: 4)를 곱해주는 MulOp 명령어를 추가합니다 (스케일링).

// 그 후 r1 + (스케일링된 r2)를 수행합니다.

| (정수, 포인터) ->

// 위와 마찬가지로 정수 쪽(r1)을 스케일링하고 더합니다.

| _ ->

// 일반 정수 덧셈입니다. 그냥 AddOp를 씁니다.

이렇게 getTyp이라는 나침반을 하나 만들어두면, Add 뿐만 아니라 Sub, Assign 등 타입을 알아야 하는 다른 곳에서도 아주 유용하게 쓸 수 있을 겁니다.

이 구조로 한번 구현해 보시겠어요? 막히면 또 물어보세요! 화이팅! 🚀

3-1. 내가 입력한 Prompt (배열 접근과 L-value 처리)

5번째 테스트 케이스(prog-5)에서 자꾸 실패해. 코드를 분석해보니 int arr[3];을 선언하고

`p = arr;` 할 때랑 `arr[0] = x;` 할 때 처리가 헛갈려.

`Var x`를 처리할 때, `x`가 일반 정수 변수일 때는 `LoadWord`로 값을 가져오는데, 배열 이름 (`arr`)일 때는 값을 로드하면 안 되고 주소 자체를 써야 하잖아. 이걸 `transExp`에서 어떻게 구분해서 처리해야 해?

3-2. LLM이 출력한 답변

매우 정확하게 파악하셨습니다! 그 부분이 바로 ****Array Decay(배열이 포인터로 변환되는 현상)****를 구현할 때 가장 핵심이 되는 부분입니다.

말씀하신 대로, `Var x`를 처리할 때 `x`가 ****일반 변수(Scalar)****인지 ****배열(Array)****인지에 따라 동작이 달라져야 합니다.

1. 일반 변수 (`int x`): 메모리 주소(`addr`)에 있는 값을 읽어와야 하므로 `LoadWord`를 써야 합니다.
2. 배열 변수 (`int arr[3]`): 식(`Expression`)에서 배열 이름은 배열의 시작 주소로 취급되어야 합니다. 이미 `addr`가 그 시작 주소를 담고 있으므로, 값을 로드하지 않고 주소(`addr`)를 그대로 반환해야 합니다.

이를 구현하기 위해 `transExp` 내부의 `Var x` 패턴 매칭에서 ****심볼 테이블에 저장된 타입(`typ`)****을 확인하여 분기 처리를 하면 됩니다.

해결 방법 (코드 예시)

`transExp` 함수의 `Var x` 부분을 아래와 같이 수정해 보세요.

F#

```
| Var x ->
```

```
    // 1. 심볼 테이블에서 변수의 주소(addr)와 타입(typ)을 가져옵니다.
```

```
    let (addr, typ) = Map.find x symtab
```

```
    match typ with
```

```
    // 2. 타입이 배열(CIntArr, CBoolArr)인 경우:
```

```
    //    값을 로드하지 않고, 주소 레지스터(addr)를 그대로 반환합니다. (Array Decay)
```

```
    | CIntArr _ | CBoolArr _ -> (addr, [])
```

```
// 3. 그 외의 경우 (int, bool, int*, bool* 등):
//     메모리에서 값을 로드(Load)하는 명령어를 생성합니다.
| _ ->
    let r = createRegName ()
    let instr =
        if sizeof typ = 4 then LoadWord (r, addr)
        else LoadByte (r, addr)
    (r, [instr])
```

추가 팁: Arr (x, idx) 처리 (arr[0] vs p[0])

이 로직은 Arr (배열 접근, x[i])을 구현할 때도 비슷하게 적용됩니다.

- x가 배열(int arr[3])이면: symtab에 있는 주소(addr)가 곧 배열의 시작 주소이므로 그대로 베이스 주소로 씁니다. (Set으로 복사)
- x가 포인터(int *p)면: p는 주소값을 담고 있는 변수이므로, LoadWord를 통해 p가 가리키는 실제 주소 값을 먼저 꺼내와야 합니다.

이 두 가지를 구분해서 처리하면 5번 테스트 케이스의 p = arr;와 arr[0] = x;를 모두 올바르게 변환할 수 있습니다.

이해되셨나요? 이 부분을 적용해서 다시 구현해 보세요! 🚀

4-1. 내가 입력한 Prompt (단락 평가 구현)

And (&&)랑 Or (||) 연산자를 구현해야 하는데, 단순히 BinOp로 처리하면 안 된다고 했어. 앞의 결과에 따라 뒤의 식을 실행하지 말아야 하는데(Short-circuit), 이걸 IR 레벨에서 어떻게 구현해야 해? Label이랑 Goto를 써야 할 것 같은데 구조를 잡아줘.

4-2. LLM이 출력한 답변

논리 연산자는 값 계산보다는 **제어 흐름(Control Flow)**으로 구현해야 합니다. Helper.createLabel()을 사용하여 라벨을 생성하고 Gotof 계열 명령어를 사용하세요.

구조 예시 (And 연산 e1 && e2):

1. falseLabel과 endLabel을 생성합니다.
2. e1을 번역한 코드를 배치하고, 결과가 0(거짓)이면 falseLabel로 점프(GotoIfNot)합니다.
3. 그 뒤에 e2를 번역한 코드를 배치하고, 결과가 0이면 falseLabel로 점프합니다.
4. 여기까지 도달했다면 둘 다 참이므로 결과 레지스터에 1을 저장하고 endLabel로 점프합니다.
5. falseLabel 위치에 결과 레지스터에 0을 저장하는 코드를 둡니다.
6. 마지막으로 endLabel을 배치합니다.

이렇게 하면 e1이 거짓일 때 e2를 실행하는 명령어들을 건너뛰게 됩니다.

이 구조를 사용하면 ****단락 평가(Short-circuit evaluation)****를 완벽하게 구현할 수 있습니다! 이해가 되셨나요? 바로 적용해 보세요! 🚀

5-1. 내가 입력한 Prompt (실행 및 채점 문제)

```
cse20211605@cspro2:~/Prj2/Prj2$ ./check.py
```

```
bash: ./check.py: Permission denied
```

check.py를 실행하려고 하는데 권한 거부 오류가 나. 그리고 빌드하고 나서 기존 빌드 파일을 지우는 make clean 같은 건 없어?

5-2. LLM이 출력한 답변

리눅스 환경에서 흔히 겪을 수 있는 권한 문제와 .NET 빌드 시스템에 대한 질문이시군요. 바로 해결 방법을 알려드리겠습니다.

1. check.py 권한 거부 (Permission denied) 해결

리눅스에서는 스크립트 파일을 실행하려면 ****실행 권한(Execute Permission, x)****이 있어야 합니다. 파일을 복사해오거나 압축을 풀면 이 권한이 빠져있는 경우가 많습니다.

다음 명령어를 입력하여 권한을 부여하세요:

Bash

```
chmod +x check.py
```

- **설명:** chmod는 파일의 권한을 변경하는 명령어이며, +x는 실행 권한을 추가하는 뜻입니다.

- **팁:** 만약 다른 스크립트나 실행 파일도 권한 문제가 생긴다면, 현재 폴더의 모든 파일에 권한을 주는 `chmod -R +x .` 명령어를 사용해도 됩니다.
-

2. make clean (빌드 파일 삭제) 방법

이번 프로젝트는 make가 아닌 **.NET (dotnet)** 빌드 시스템을 사용하기 때문에 make clean 명령어는 동작하지 않습니다.

강의 자료에서 `dotnet build -o out` 명령어를 사용했으므로, 빌드 결과물은 모두 **out** 폴더에 모여 있습니다. 따라서 가장 깔끔하게 청소하는 방법은 **out** 폴더를 삭제하는 것입니다.

Bash

```
rm -rf out
```

- **설명:** rm은 삭제 명령어이며, -rf는 디렉토리 내부까지 강제로 삭제하라는 옵션입니다.
- **참고:** 만약 bin이나 obj 폴더가 생성되었다면 그것들도 함께 지워주시면 프로젝트를 완전히 초기화할 수 있습니다 (`rm -rf out bin obj`).

이제 권한을 주고 채점 스크립트를 다시 실행해 보세요! 🚀